

Exercise 2 - Random Number Generation through CDF and acceptance-rejection sampling

Florescu Andrei - 12242987

2024

1. Linear Congruential Random Number Generation Algorithm

```
mc.gen <- function(n, m, a, c = 0, x0) {  
  us <- numeric(n)  
  for (i in 1:n) {  
    x0 <- a * (x0 + c) %% m  
    us[i] <- x0 / m  
  }  
  return(us)  
}
```

```
seed <- 12242987
```

```
round(mc.gen(8, 17, 3, 0, seed), 4)
```

```
## [1] 2.1176 0.3529 1.0588 0.1765 0.5294 1.5882 1.7647 2.2941
```

```
round(mc.gen(15, 11, 3, 0, seed), 4)
```

```
## [1] 2.4545 1.3636 1.0909 0.2727 0.8182 2.4545 1.3636 1.0909 0.2727 0.8182
```

```
## [11] 2.4545 1.3636 1.0909 0.2727 0.8182
```

```
round(mc.gen(15, 12451, 110, 0, seed), 4)
```

```
## [1] 32.2817 30.9919 109.1077 11.8472 93.1965 21.6183 68.0178 1.9613
```

```
## [9] 105.7417 81.5878 64.6607 72.6737 74.1049 11.5380 59.1832
```

```
round(mc.gen(15, 9, 3, 0, seed), 4)
```

```
## [1] 2.6667 2.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
```

```
## [11] 0.0000 0.0000 0.0000 0.0000 0.0000
```

```
round(mc.gen(15, 10, 3, 0, seed), 4)
```

```
## [1] 2.1 0.3 0.9 2.7 2.1 0.3 0.9 2.7 2.1 0.3 0.9 2.7 2.1 0.3 0.9
```

We can see that for certain values of m and a the randomly generated samples either repeat, or either they have very predictable values (i.e. with one decimal).

2. Exponential Distribution

Mathematical formula

Using the inverse transform sampling method we can solve for X in the exponential Distribution formula. Therefore the formula becomes:

$$F(x) = 1 - \exp(-\lambda x)$$

We want the generated number to be uniform (between 0 and 1)

$$U = F(X) = 1 - \exp(-\lambda X)$$

$$\exp(-\lambda X) = 1 - U$$

Apply natural logarithm on both sides.

$$X = -\frac{1}{\lambda} \ln(1 - U)$$

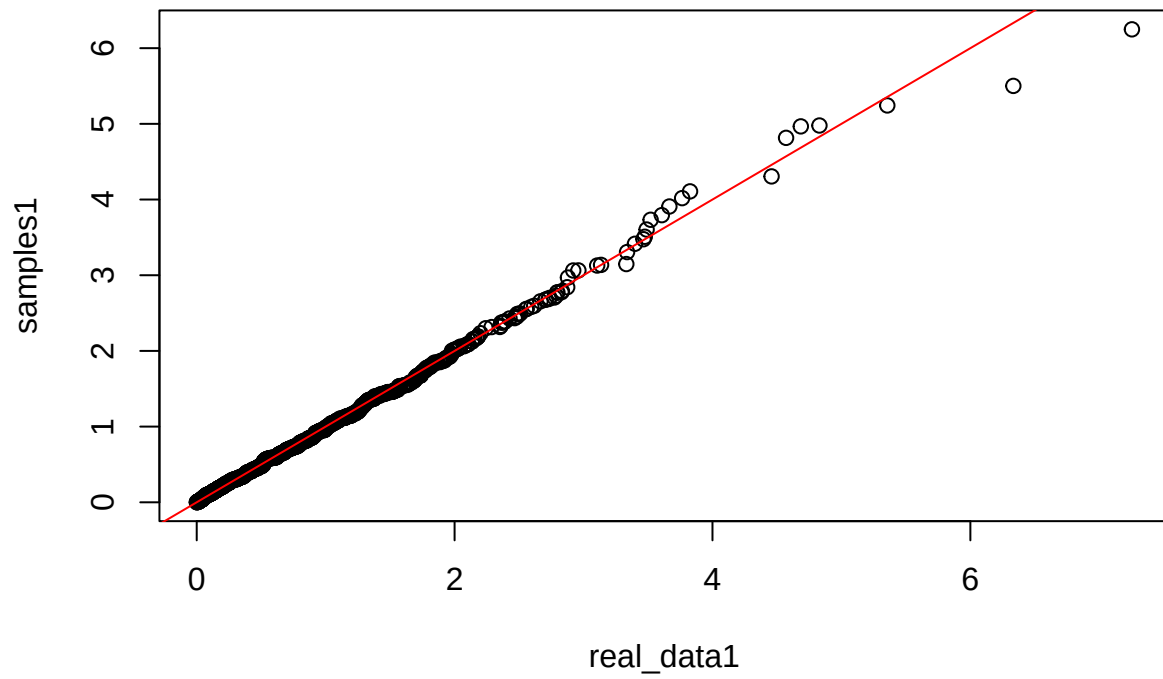
Since U is between 0 and 1 we can rewrite the formula without 1 - U since its the same

$$X = -\frac{1}{\lambda} \ln(U)$$

R implementation

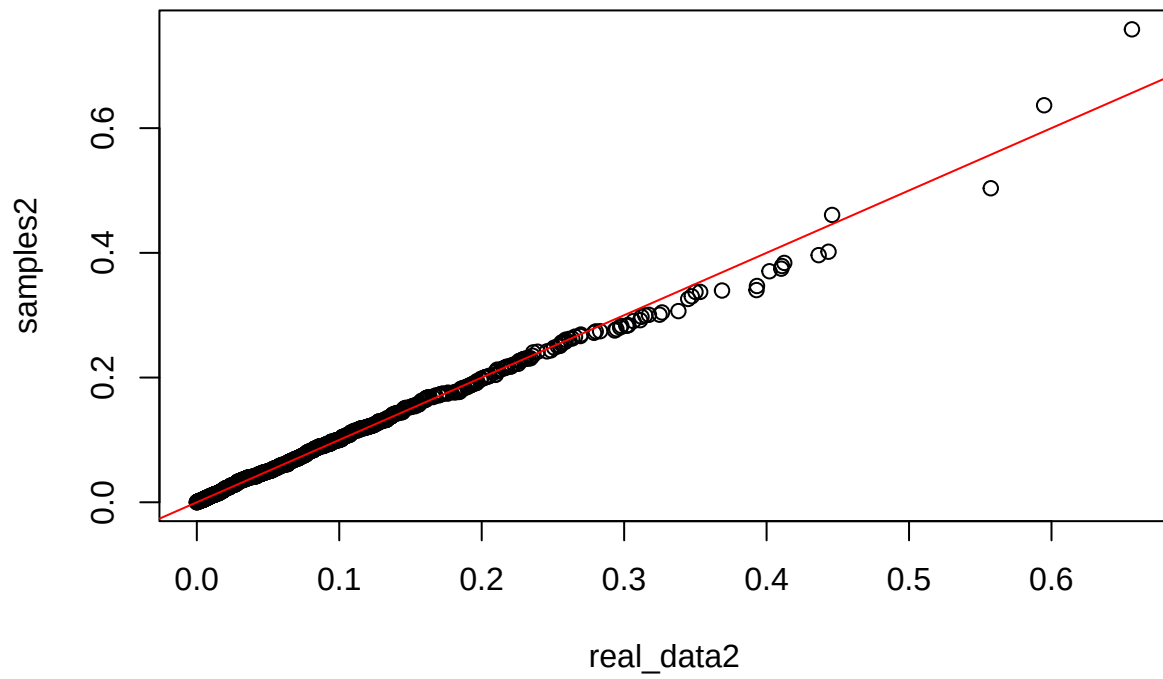
```
exp.rng <- function(n, lambda) {  
  X <- numeric(n)  
  uniform_vars <- runif(n)  
  for (i in 1:n) {  
    X[i] <- -1.0 / lambda * log(uniform_vars[i])  
  }  
  return(X)  
}  
  
samples1 <- exp.rng(1000, 1.2345)  
samples2 <- exp.rng(1000, 12.345)  
samples3 <- exp.rng(1000, 33.333)  
real_data1 <- rexp(1000, 1.2345)  
real_data2 <- rexp(1000, 12.345)  
real_data3 <- rexp(1000, 33.333)  
  
qqplot(real_data1, samples1, main = "QQ-Plot for lambda = 0.12345")  
abline(0, 1, col = "red")
```

QQ-Plot for lambda = 0.12345



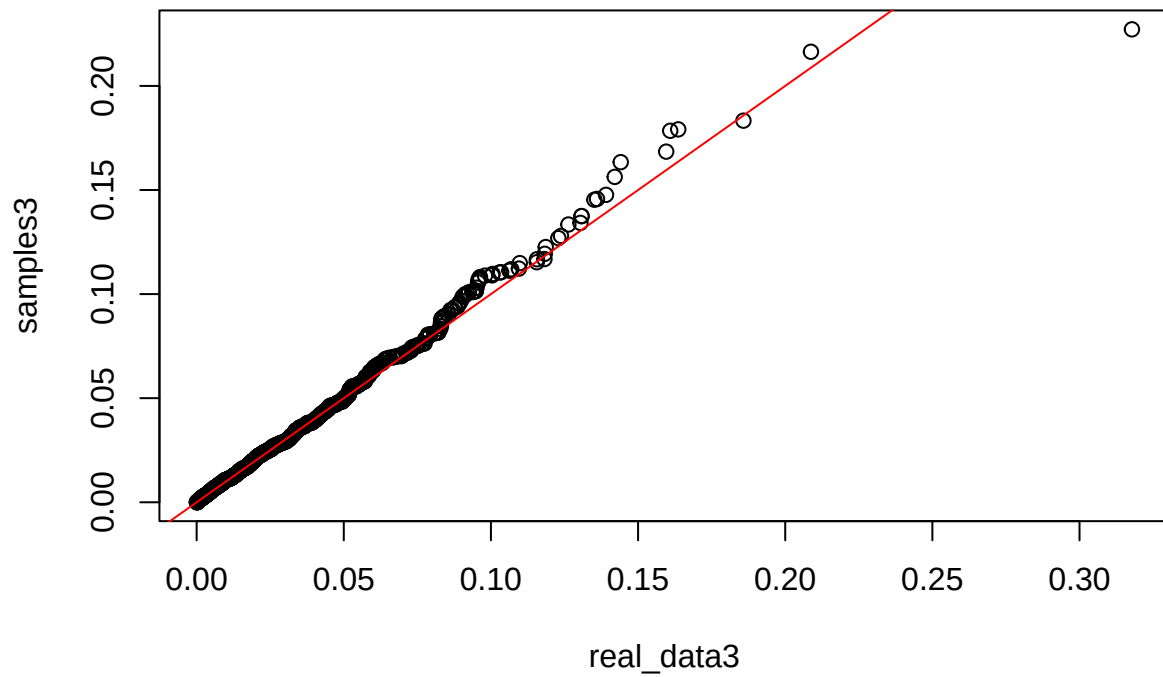
```
qqplot(real_data2, samples2, main = "QQ-Plot for lambda = 1234.5")  
abline(0, 1, col = "red")
```

QQ-Plot for lambda = 1234.5



```
qqplot(real_data3, samples3, main = "QQ-Plot for lambda = 33.333")  
abline(0, 1, col = "red")
```

QQ-Plot for lambda = 33.333

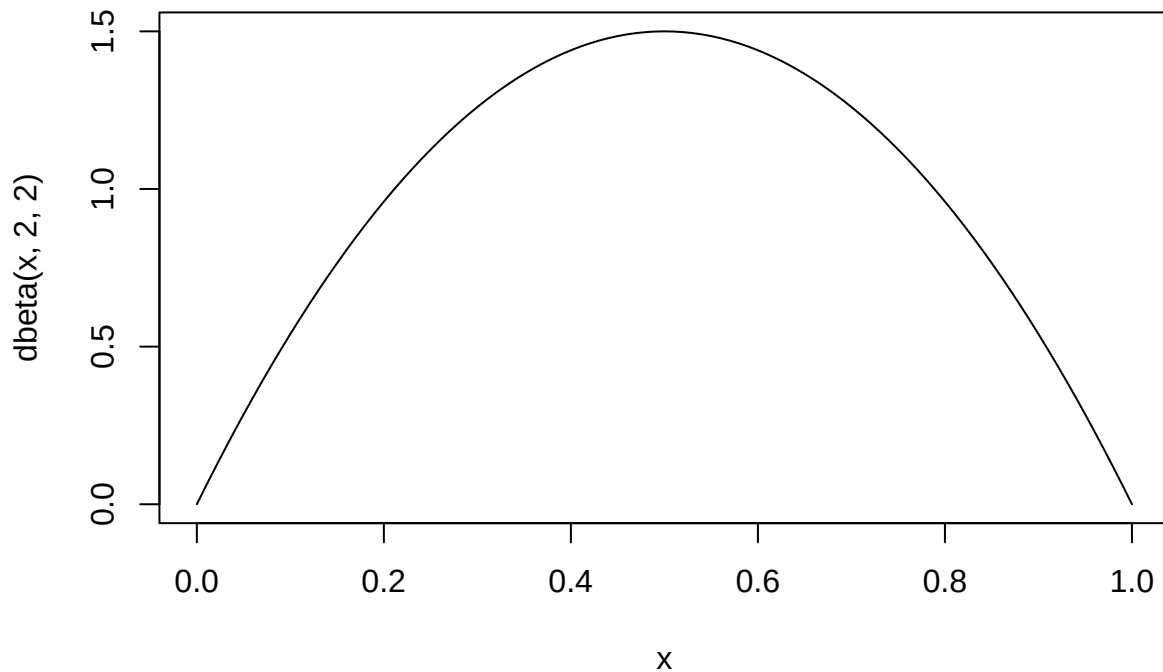


3. Beta Distribution

Beta function where alpha and beta are 2

For the beta distribution where alpha and beta are 2, the function is really close to a uniform distribution.

```
curve(dbeta(x, 2, 2), from = 0, to = 1)
```



We can see that the maximum value is for $x = 0.5$

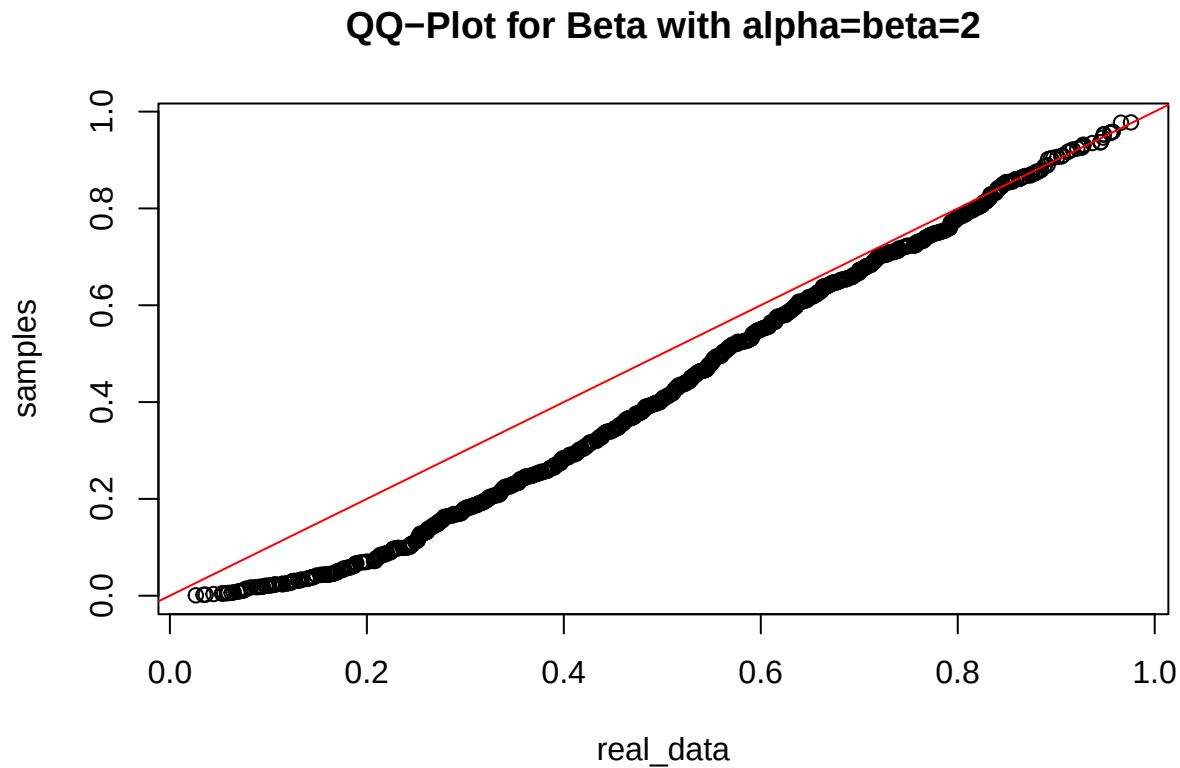
```
dbeta(0.5, 2, 2)
```

```
## [1] 1.5
```

Therefore 1.5 is a good constant to use with the uniform distribution for alpha and beta equal to 2

```
beta.rng22 <- function(n) {  
  x <- numeric(n)  
  accepted <- 0  
  iter <- 0  
  c <- 1.5  
  while (accepted < n) {  
    u <- runif(1)  
    iter <- iter + 1  
    y <- runif(1)  
    f <- dbeta(y, 2, 2)  
  
    if (u <= f / (c * y)) {  
      accepted <- accepted + 1  
      x[accepted] <- y  
    }  
  }  
  return(x)  
}  
  
samples <- beta.rng22(1000)
```

```
real_data <- rbeta(1000, 2, 2)
qqplot(real_data, samples, main = "QQ-Plot for Beta with alpha=beta=2")
abline(0, 1, col = "red")
```



Generalization

For a function that can have alpha and beta greater than one, we can still sample from the uniform distribution. The maximum value is always going to be at $x=0.5$ therefore we can calculate the constant at the beginning of the function as $c=\text{beta}(0.5, \alpha, \beta)$.

```
beta.rng <- function(n, alpha, beta) {
  x <- numeric(n)
  accepted <- 0
  iter <- 0
  c <- dbeta(0.5, alpha, beta)
  while (accepted < n) {
    u <- runif(1)
    iter <- iter + 1
    y <- runif(1)
    f <- dbeta(y, alpha, beta)

    if (u <= f / (c * y)) {
      accepted <- accepted + 1
      x[accepted] <- y
    }
  }
}
```

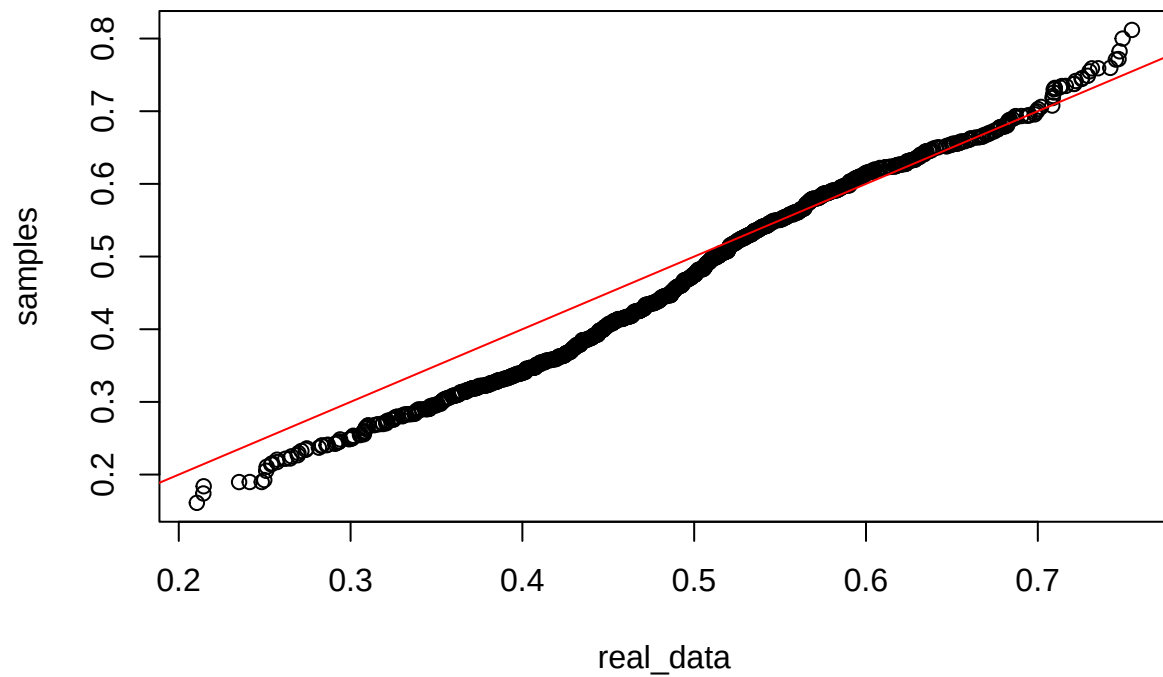
```

    return(x)
}

samples <- beta.rng(1000, 10, 10)
real_data <- rbeta(1000, 10, 10)
qqplot(real_data, samples, main = "QQ-Plot for Beta with alpha=beta=10")
abline(0, 1, col = "red")

```

QQ-Plot for Beta with alpha=beta=10

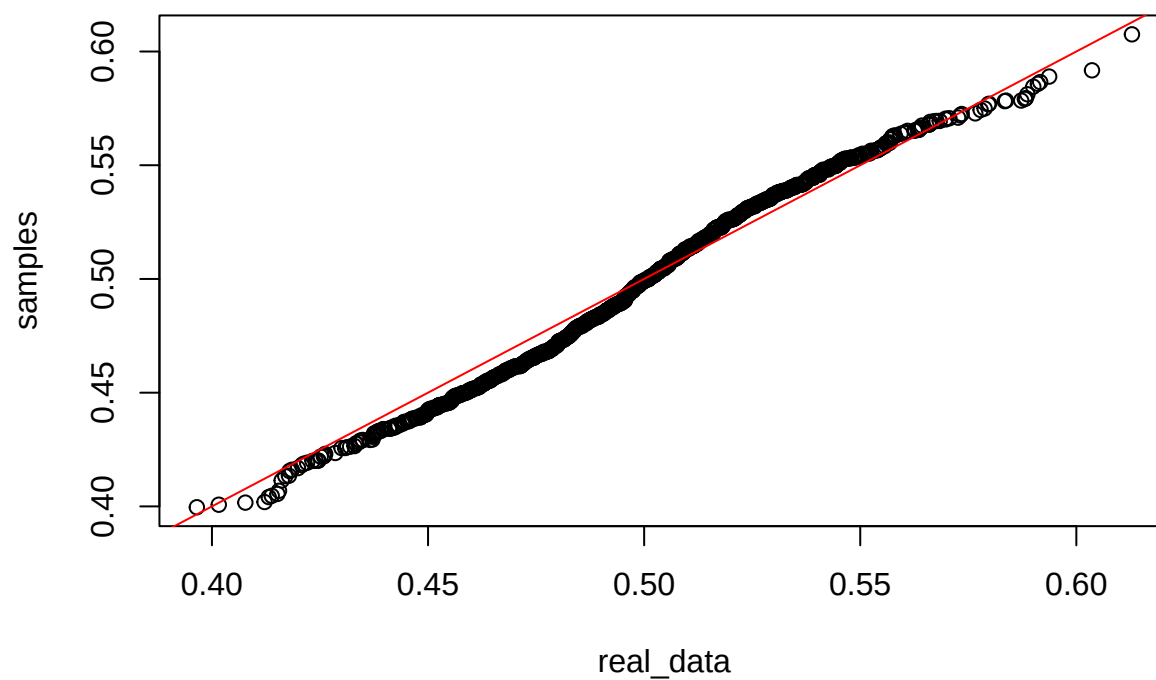


```

samples <- beta.rng(1000, 100, 100)
real_data <- rbeta(1000, 100, 100)
qqplot(real_data, samples, main = "QQ-Plot for Beta with alpha=beta=100")
abline(0, 1, col = "red")

```

QQ-Plot for Beta with alpha=beta=100



One could argue that we could use the normal distribution when alpha and beta are very large because in the Relations between Distributions visualised slide in the course, for alpha and beta closing in on infinity, the beta distribution is closer to the normal distribution.